Primjena MinHash i LSH algoritama za pronalaženje sličnih dokumenata

SARA DAUTBEGOVIĆ, Elektrotehnički fakultet Univerziteta u Sarajevu, Bosna i Hercegovina
JASMINA PILAV*, Elektrotehnički fakultet Univerziteta u Sarajevu, Bosna i Hercegovina

U radu je implementiran sistem za pronalaženje sličnih dokumenata zasnovan na MinHash i Locality-Sensitive Hashing (LSH) pristupu. Dokumenti se predstavljaju kao skupovi shinglova, a sličnost se posmatra kroz Jaccardov koeficijent. MinHash potpisi omogućavaju efikasnu aproksimaciju Jaccard sličnosti, dok LSH organizuje potpise u buckete kako bi se pretraga svela na mali skup kandidata umjesto poređenja sa cijelom kolekcijom. Eksperimentalna evaluacija na podskupovima do 20 000 dokumenata pokazuje da LSH ostvaruje višeredno ubrzanje u odnosu na brute-force pretragu, uz zadržavanje praktično korisnog kvaliteta vraćenih kandidata (mjereno prosječnom stvarnom sličnosti i hit-at-topK metrikom).

1 Uvod

Pronalaženje sličnih tekstova u velikim skupovima podataka predstavlja značajan izazov zbog eksponencijalnog rasta broja dokumenata. Tradicionalni pristupi zahtijevaju poređenje svakog para dokumenata, što dovodi do kvadratne vremenske složenosti. U ovom radu implementiran je sistem za pronalaženje sličnih vijesti korištenjem MinHash i Locality-Sensitive Hashing (LSH) algoritama. Slični problemi pronalaženja sličnih obrazaca i deduplikacije u velikim skupovima podataka razmatraju se i u klasičnoj literaturi iz oblasti data mininga [3]. Korištenjem sentence embeddinga postiže se bolje semantičko predstavljanje tekstova. Rezultati pokazuju da LSH značajno smanjuje broj poređenja uz zadržavanje zadovoljavajuće tačnosti.

2 Teorijski uvod

2.1 Sličnost dokumenata i Jaccardov indeks

Pronalaženje sličnosti između dokumenata predstavlja osnovni problem u analizi velikih skupova tekstualnih podataka, posebno u oblastima kao što su pretraga informacija, detekcija duplikata i analiza vijesti. Uobičajen pristup, koji su formalizovali Broder i saradnici, zasniva se na predstavljanju dokumenta kao skupa karakterističnih elemenata. Ti elementi mogu biti riječi, n-grami ili tzv. *shingles*, koji predstavljaju uzastopne podnizove teksta fiksne dužine [1].

Kada su dokumenti predstavljeni kao skupovi, njihova sličnost se prirodno mjeri pomoću Jaccard koeficijenta [1]. Jaccard sličnost između dva skupa A i B definiše se kao odnos broja zajedničkih elemenata i ukupnog broja jedinstvenih elemenata u oba skupa:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \tag{1}$$

.

Vrijednost Jaccard koeficijenta nalazi se u intervalu [0,1], gdje veće vrijednosti ukazuju na veću sličnost između dokumenata. Ova mjera je intuitivna i dobro prilagođena problemu poređenja skupova, zbog čega se često koristi kao referentna mjera sličnosti u analizi dokumenata. Zbog svoje jednostavnosti i interpretabilnosti, Jaccard sličnost se široko koristi u sistemima za pretragu informacija i analizu dokumenata [2].

Međutim, direktno računanje Jaccard sličnosti nad velikim skupovima dokumenata predstavlja ozbiljan računski problem. Za skup dokumenata veličine n, poređenje svakog para dokumenata zahtijeva $O(n^2)$ operacija, što postaje neizvodljivo u realnim sistemima sa velikim brojem tekstova. Ovaj problem motivisao je razvoj aproksimativnih metoda koje omogućavaju efikasno procjenjivanje sličnosti bez eksplicitnog poređenja svih elemenata skupova.

.

2.2 Princip MinHash algoritma i min-wise nezavisno hashiranje

MinHash algoritam, koji su predstavili Broder i saradnici, razvijen je kao efikasna metoda za aproksimaciju Jaccard sličnosti između velikih skupova. Osnovna ideja MinHash-a zasniva se na konceptu min-wise nezavisnih permutacija elemenata skupa. Ako se elementi univerzalnog skupa permutuju slučajnim redoslijedom, vjerovatnoća da dva skupa imaju isti minimalni element nakon permutacije jednaka je njihovoj Jaccard sličnosti [1].

Formalno, za dva skupa A i B, vjerovatnoća da je minimalni element u permutovanom redoslijedu isti za oba skupa može se izraziti kao [1]:

$$\Pr\left[\min\left(\pi(A)\right) = \min\left(\pi(B)\right)\right] = J(A, B) \tag{2}$$

gdje π označava slučajnu permutaciju elemenata univerzalnog skupa. Ova osobina omogućava da se Jaccard sličnost aproksimira posmatranjem samo minimalnih vrijednosti, umjesto kompletnog sadržaja skupova.

U praksi, generisanje i primjena stvarnih slučajnih permutacija nad velikim univerzalnim skupovima je računski preskupo. Zbog toga se umjesto eksplicitnih permutacija koriste hash funkcije koje aproksimiraju ponašanje min-wise nezavisnih permutacija. Svaka hash funkcija preslikava elemente skupa u numeričke vrijednosti, a MinHash vrijednost dokumenta dobija se kao minimalna hash vrijednost među svim elementima tog dokumenta.

Korištenjem više nezavisnih hash funkcija, za svaki dokument se formira MinHash potpis (signature) koji predstavlja sažetu reprezentaciju originalnog skupa. Sličnost između dva dokumenta tada se procjenjuje kao udio hash funkcija za koje oba dokumenta imaju istu MinHash vrijednost. Ova aproksimacija je efikasna, jer značajno smanjuje količinu podataka potrebnih za poređenje, dok istovremeno zadržava statističku povezanost sa stvarnom Jaccard sličnosti.

Min-wise nezavisne hash funkcije na ovaj način omogućavaju efikasnu i skalabilnu aproksimaciju sličnosti dokumenata, bez potrebe za eksplicitnim poređenjem svih elemenata skupova, što MinHash čini pogodnim za primjenu u velikim i dinamičnim skupovima tekstualnih podataka.

2.3 Univerzalno hashiranje (Carter–Wegman familija)

Koncept univerzalnog hashiranja uveli su Carter i Wegman kao teorijski okvir za konstrukciju familija hash funkcija sa dobrim probabilističkim svojstvima [6]. Ideja univerzalnog hashiranja zasniva se na izboru hash funkcije iz unaprijed definisane familije funkcija na slučajan način, čime se smanjuje vjerovatnoća sudara (kolizija) između različitih ulaznih vrijednosti [5].

Formalno, familija hash funkcija $\mathcal{H}$ naziva se univerzalnom ako za svaka dva različita elementa $x \neq y$ važi:

$$\Pr_{h \in \mathcal{H}}\left[h(x) = h(y)\right] \leq \frac{1}{p} \tag{3}$$

gdje je p veličina hash prostora.

U praksi, često korištena i efikasna instanca univerzalne familije hash funkcija ima linearni oblik:

$$h(x) = (ax + b) \bmod p \tag{4}$$

gdje su koeficijenti a i b slučajno izabrani cijeli brojevi iz intervala $[1, p-1]$, dok je p prost broj veći od maksimalne vrijednosti ulaznih podataka. U implementaciji MinHash algoritma, nizovi $A[i]$ i $B[i]$ predstavljaju upravo ove slučajno generisane koeficijente, pri čemu svaka kombinacija $(A[i], B[i])$ definiše jednu nezavisnu hash funkciju.

Ovakav pristup omogućava generisanje velikog broja hash funkcija uz minimalan računski trošak, bez potrebe za eksplicitnim konstruisanjem slučajnih permutacija nad velikim univerzalnim skupovima. Carter–Wegman univerzalno hashiranje na taj način predstavlja efikasnu i teorijski

utemeljenu zamjenu za permutacije, što ga čini posebno pogodnim za primjenu u MinHash algoritmu nad velikim skupovima podataka.

2.4 Efikasno hashiranje stringova za MinHash

U kontekstu MinHash algoritma, primjena kriptografskih hash funkcija poput MD5 ili SHA nije praktična zbog njihovog visokog računskog troška. Ove funkcije su dizajnirane sa ciljem sigurnosti i otpornosti na napade, a ne brzine, što ih čini neefikasnim za obradu velikog broja tokena u realnom vremenu.

Lemire i Kaser u svom radu pokazuju da se snažno univerzalno hashiranje stringova može ostvariti korištenjem jednostavnih i brzih hash funkcija, uz primjenu linearnih transformacija [5]. Umjesto kriptografskih hashova, koristi se brzo hashiranje stringova koje proizvodi cjelobrojne vrijednosti, nad kojima se zatim primjenjuju univerzalne hash funkcije.

U praksi, postupak se sastoji iz dvije faze:

(1) brzo hashiranje tekstualnih tokena u cjelobrojne (integer) vrijednosti,
(2) primjena univerzalnih hash funkcija nad dobijenim integerima.

Ovakva kombinacija omogućava postizanje dobre raspodjele hash vrijednosti uz znatno manji računski trošak. Implementacija MinHash algoritma koristi upravo ovaj princip, gdje se tokeni dokumenata prvo mapiraju na numeričke vrijednosti, a zatim se nad njima primjenjuju linearne hash funkcije iz Carter–Wegman familije.

Na ovaj način postiže se efikasnost potrebna za obradu velikih tekstualnih skupova, uz zadržavanje probabilističkih garancija koje su neophodne za pouzdanu aproksimaciju Jaccard sličnosti.

2.5 Locality-Sensitive Hashing (LSH)

Locality-Sensitive Hashing (LSH) predstavlja tehniku za brzo pronalaženje sličnih dokumenata, zasnovanu na MinHash potpisima [4]. Osnovna ideja LSH-a jeste da se slični dokumenti sa velikom vjerovatnoćom mapiraju u iste „buckete“, dok se neslični dokumenti razdvajaju.

Nakon generisanja MinHash potpisa, potpisi se organizuju u matricu potpisa (signature matrix), gdje svaki red odgovara jednoj hash funkciji, a svaka kolona jednom dokumentu. Ova matrica se zatim dijeli na b pojaseva (bands), pri čemu svaki pojas sadrži r redova, tako da važi:

$$k = b \cdot r \tag{5}$$

, gdje je k ukupan broj hash funkcija.

Za svaki pojas, vrijednosti u odgovarajućim redovima se kombinuju i hashiraju u „bucket“. Dva dokumenta postaju kandidati za sličnost ako završe u istom bucketu barem u jednom pojasu. Vjerovatnoća da dva dokumenta sa Jaccard sličnosti s postanu kandidati data je izrazom [4]:

$$P(s) = 1 - (1 - s^r)^b \tag{6}$$

Primjenom LSH-a izbjegava se direktno poređenje svih parova dokumenata, čija vremenska složenost iznosi $O(n^2)$, umjesto čega se poređenje svodi samo na dokumente koji završe u istim bucketima, čime se postiže značajno smanjenje broja poređenja. Zbog svoje probabilističke prirode, LSH ne garantuje pronalazak svih sličnih parova, ali u praksi pruža izuzetno dobar kompromis između tačnosti i efikasnosti, što ga čini standardnim rješenjem za pronalaženje sličnosti u velikim skupovima podataka.

3 Implementacija algoritma

U ovom poglavlju opisana je implementacija algoritma za pronalaženje sličnih dokumenata korištenjem MinHash i Locality-Sensitive Hashing (LSH) pristupa. Opis je fokusiran isključivo na

algoritamske korake i način njihove realizacije, dok su skup podataka i eksperimentalna evaluacija obrađeni u posebnom poglavlju.

3.1 Hashiranje elemenata i univerzalne hash funkcije

Svaki dokument se prvo predstavlja kao skup tekstualnih elemenata (shinglova). Kako bi se izbjeglo skupo generisanje eksplicitnih permutacija, svaki shingle se najprije preslikava u 64-bitnu cjelobrojnu vrijednost korištenjem brzog hashiranja. Nad tim vrijednostima se zatim primjenjuje familija univerzalnih hash funkcija Carter–Wegman tipa oblika

$$h_i(x) = (a_i \cdot x + b_i) \bmod p,$$

gdje su koeficijenti a_i i b_i slučajno generisani, a p je veliki prost broj.

Ovaj pristup predstavlja efikasnu aproksimaciju min-wise nezavisnih permutacija uz minimalan računski trošak.

3.2 Generisanje MinHash potpisa

MinHash potpis dokumenta dobija se tako što se za svaku hash funkciju zadržava minimalna hash vrijednost dobijena nad svim shinglovima dokumenta. Implementacija koristi vektorizovano računanje kako bi se minimizirao broj operacija i ubrzalo izvršavanje.

Algorithm 1: Generisanje MinHash potpisa

```
Input: Skup shinglova dokumenta S, koeficijenti a[1..k], b[1..k], prost broj p
Output: MinHash potpis signature[1..k]
for i ← 1 to k do
    signature[i] ← p;
foreach shingle s ∈ S do
    x ← baseHash(s);
    for i ← 1 to k do
        h ← (a[i] · x + b[i]) mod p;
        signature[i] ← min(signature[i], h);
return signature
```

Ovim postupkom se svaki dokument preslikava u fiksni vektor cjelobrojnih vrijednosti, čija statistička svojstva odražavaju sličnost originalnih skupova.

3.3 Aproksimacija Jaccard sličnosti

Sličnost između dva dokumenta procjenjuje se poređenjem njihovih MinHash potpisa. Aproksimirana Jaccard sličnost računa se kao udio pozicija na kojima se vrijednosti potpisa podudaraju.

$$\widehat{J}(A, B) = \frac{1}{k} \sum_{i=1}^{k} \mathbb{I}\left[signature_A[i] = signature_B[i]\right].$$

Ova mjera predstavlja nepristrasnu procjenu stvarne Jaccard sličnosti između skupova shinglova.

3.4 Locality-Sensitive Hashing (LSH)

Kako bi se izbjeglo poređenje svih parova dokumenata, MinHash potpisi se dodatno obrađuju korištenjem LSH tehnike. Potpis se dijeli na b traka (bands), pri čemu svaka traka sadrži r uzastopnih redova. Svaka traka se hashira u poseban bucket.

Algorithm 2: Formiranje LSH bucketa

Input: MinHash potpisi, broj traka b, broj redova r
Output: LSH bucketi
foreach *dokument* d **do**
  for $j \leftarrow 1$ **to** b **do**
    $band \leftarrow$ izdvoji r redova potpisa dokumenta d;
    $bucket \leftarrow hash(band, j)$;
    dodaj d u bucket $(j, bucket)$;

Dokumenti koji završe u istom bucketu u barem jednoj traci smatraju se kandidatima za sličnost.

3.5 Pronalaženje kandidata

Za upitni dokument, kandidati se pronalaze tako što se njegov MinHash potpis obrađuje na isti način kao i potpisi u fazi izgradnje, te se prikupljaju svi dokumenti koji dijele barem jedan zajednički bucket.

Algorithm 3: Pronalaženje kandidata pomoću LSH

Input: MinHash potpis upita, LSH bucketi
Output: Skup kandidata
$C \leftarrow \emptyset$;
for $j \leftarrow 1$ **to** b **do**
  $band \leftarrow$ izdvoji r redova potpisa upita;
  $bucket \leftarrow hash(band, j)$;
  $C \leftarrow C \cup$ dokumenti iz bucketa $(j, bucket)$;
return C

Na ovaj način se broj potrebnih poređenja smanjuje sa kvadratne složenosti na znatno manji broj kandidata, uz zadržavanje visoke vjerovatnoće pronalaska stvarno sličnih dokumenata.

4 Ekperimentalna evaluacija i rezultati

4.1 Skup podataka

Skup podataka korišten u ovom radu preuzet je sa platforme Hugging Face i sastoji se od engleskih članaka iz Wikipedije (`wikimedia/wikipedia`, konfiguracija `20231101.en`). Podaci su učitani iz `train` dijela u *streaming* režimu, što omogućava obradu velikog broja dokumenata bez potpunog preuzimanja u memoriju. Svaki zapis sadrži tri kolone: `doc_id` (jedinstveni identifikator dokumenta), `title` (naslov članka) i `text` (puni tekst članka). Prije odabira ovog skupa, algoritam je inicijalno testiran na više skupova podataka, jedan je bio Kaggle skup vijesti *News Category Dataset*, ali je zbog izraženih razlika u dužini i strukturi tekstova ipak odabran Wikipedia skup koji pruža sadržajno bogatije i duže dokumente pogodnije za evaluaciju sličnosti.

4.2 Postavke i parametri

Kako bi se odabrala finalna konfiguracija MinHash+LSH sistema, provedena su **četiri eksperimenta** u kojima su sistematski mijenjane ključne hiperpostavke: broj permutacija, veličina shingla, LSH raspodjela potpisa na trake i redove te slučajni seed. Cilj je bio pronaći konfiguraciju koja daje najbolji kompromis između kvaliteta pronađenih kandidata (procijenjena sličnost) i efikasnosti (broj kandidata i vrijeme).

Rezultati su pokazali da previše "stroge" LSH konfiguracije (veći broj redova po traci, npr. $r \geq 4$) često dovode do toga da upiti ne dobiju nijednog kandidata (`avg_candidates=0`), dok konfiguracija

sa **više traka i manjim brojem redova** proizvodi stabilan broj kandidata i mjerljivo bolju prosječnu sličnost. Konkretno, u trade-off testu jedino konfiguracija sa manjim brojem redova po traci daje nenulti broj kandidata (`avg_candidates`=5.32, `median_candidates`=1) i veću prosječnu stvarnu sličnost među pronađenim parovima (`avg_true_jaccard`=0.002784), dok ostale testirane postavke iz mreže (b, r) daju `avg_true_jaccard`=0. Na osnovu ovih rezultata odabrani su **finalni parametri prikazani u Tabeli 1**, a evaluacija je provedena na `N_QUERIES = 50` nasumično izabranih upita kako bi se izmjerile metrike performansi.

Table 1. Finalno odabrani parametri za MinHash+LSH

Parametar	Vrijednost
Broj hash funkcija / permutacija (`num_perm`)	**128**
Veličina shingla (k_s)	**2**
Broj traka (`bands` = b)	**64**
Broj redova po traci (`rows` = r)	**2**
Broj upita u evaluaciji (`N_QUERIES`)	**50**
TOPK (broj vraćenih rezultata)	**10**
Slučajni seed	**42**

4.3 Performanse LSH pretrage

Poređenje režima evaluacije (`N_QUERIES=50` *vs* `N_QUERIES=500`*).* U evaluaciji su korištena dva režima, `N_QUERIES=50` i `N_QUERIES=500`. Veći broj upita ne mijenja ponašanje algoritma za pojedinačni upit, ali daje **stabilniju i pouzdaniju procjenu** metrika na nivou sistema, jer prosjek obuhvata širi spektar upita (uključujući i one sa nenultim brojem kandidata i slučajeve sa većom sličnosti). Očekivano, ukupno vrijeme evaluacije je veće za `N_QUERIES=500` nego za `N_QUERIES=50`, pošto se pretraga izvršava nad većim brojem upita.

Tabela 2 pokazuje da, sa porastom broja dokumenata, raste i `avg_cand` te `nonempty` (npr. `nonempty` za `N_QUERIES=500` raste sa 0.286 na 0.678), što znači da sistem u većim skupovima češće pronalazi bar jednog kandidata. Također, režim sa 500 upita češće daje **viši ili stabilniji** `avg_true_jac`, jer veći uzorak smanjuje varijansu procjene i povećava šansu da u prosjek uđu upiti sa sličnijim kandidatima. Ukupno, rezultati ukazuju da LSH zadržava smislen kvalitet kandidata uz vrlo niske troškove pretrage i do 20 000 dokumenata.

Table 2. Poređenje LSH evaluacije za $N_QUERIES = 50$ i $N_QUERIES = 500$ (izabrane metrike).

	$N_QUERIES = 50$			$N_QUERIES = 500$		
	avg_cand	nonempty	avg_true_jac	avg_cand	nonempty	avg_true_jac
1000	0.80	0.36	0.013861	0.956	0.286	0.012956
4000	3.08	0.42	0.019277	5.764	0.490	0.017085
10000	27.82	0.58	0.012240	11.814	0.614	0.024322
15000	16.88	0.58	0.013102	21.444	0.656	0.023601
20000	21.46	0.60	0.022900	25.458	0.678	0.025309

Trend prosječnog broja kandidata. Slika 1 prikazuje kako prosječan broj kandidata koje LSH vraća raste sa povećanjem broja dokumenata. Režim sa `N_QUERIES=500` daje stabilniji trend jer veći uzorak smanjuje uticaj ekstremnih upita, dok je kod `N_QUERIES=50` vidljiva veća varijabilnost i povremeni skokovi usljed slučajnog odabira upita. Ovakvo ponašanje je očekivano za probabilistične metode poput LSH, gdje broj kandidata zavisi od distribucije potpisa po bucketima i karakteristika odabranih upita.

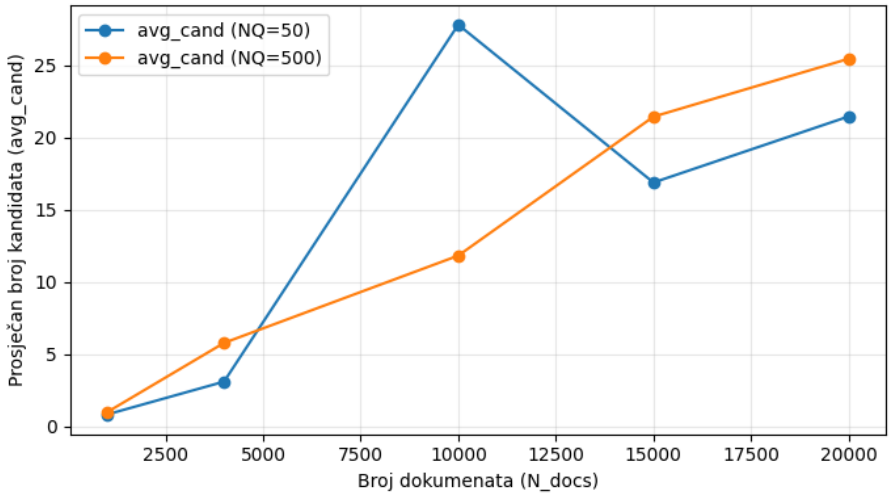

Fig. 1. Prosječan broj kandidata po upitu u zavisnosti od veličine skupa dokumenata, za `N_QUERIES=50` i `N_QUERIES=500`.

Gustoća: broj kandidata po upitu naspram prosječne sličnosti. Za razliku od prethodne evaluacije sa različitim vrijednostima `N_QUERIES`, naredna analiza je izvedena nezavisno. Hexbin prikaz je dobijen pokretanjem LSH pretrage nad nasumično odabranim podskupom od 10 000 dokumenata, kako bi se vizualizovao odnos između *broja kandidata po upitu* i *prosječne stvarne Jaccard sličnosti.* Slika 2 pokazuje da je najveća gustoća upita koncentrisana pri malom broju kandidata, što potvrđuje da LSH u većini slučajeva sužava pretragu na mali skup dokumenata.

Također, viša prosječna sličnost se uglavnom javlja kada je broj kandidata mali do umjeren, dok se pri većem broju kandidata prosjek sličnosti smanjuje. To je očekivano, jer širi skup kandidata u prosjek uključuje više slabije sličnih dokumenata. Rijetki outlieri sa veoma velikim brojem kandidata mogu ukazivati na upite sa vrlo čestim shinglovima (npr. opšte fraze).

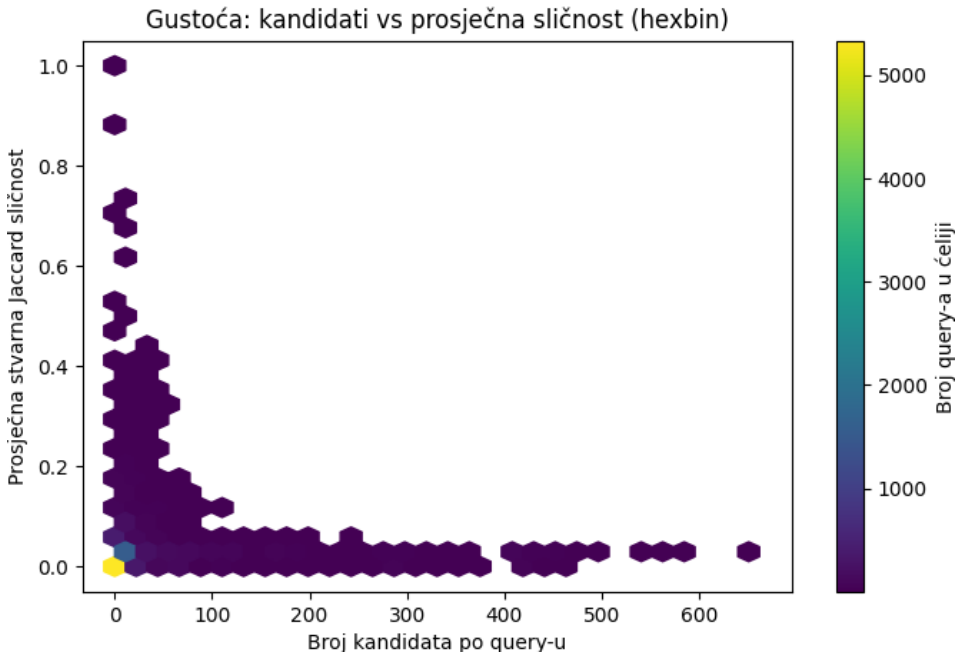

Fig. 2. Gustoća upita: broj kandidata po upitu naspram prosječne stvarne Jaccard sličnosti (hexbin).

4.4 Poređenje LSH i brute-force pretrage

Složenost (Big-O).. Neka je N broj dokumenata u kolekciji, k broj MinHash permutacija (`num_perm`), a b i r broj traka i redova po traci (pri čemu važi $k = b \cdot r$). Za jedan upit, **brute-force** pristup računa sličnost upitnog dokumenta sa *svim* dokumentima u skupu, pa mu je vremenska složenost

$$T_{\mathrm{BF}} = O(N),$$

(uz dodatni faktor cijene računanja sličnosti, npr. Jaccard nad shinglovima).
Kod **LSH** pristupa, upitni potpis se podijeli na b traka i za svaku traku se uradi lookup u bucket, što je tipično

$$T_{\mathrm{LSH}} = O(b + C),$$

gdje je C broj kandidata koje LSH vrati za taj upit. Pošto se sličnost računa samo nad kandidatima, a u praksi važi $C \ll N$, LSH postiže višestruko manju cijenu pretrage u odnosu na brute-force.

Poređenje vremena izvršavanja. Tabela 3 prikazuje poređenje LSH pretrage zasnovane na MinHash potpisima i referentnog brute-force pristupa. Brute-force za svaki upit računa sličnost prema svim dokumentima u podskupu, dok LSH najprije generiše *kandidate* putem bucketa i zatim provjerava samo taj znatno manji skup. Eksperiment je proveden nad podskupovima veličine $N_{\mathrm{docs}} \in \{1000, 4000, 10000, 15000, 20000\}$, uz fiksne parametre sistema i evaluaciju na $N_QUERIES = 50$ upita (TOPK=10).
Rezultati pokazuju da LSH vrijeme ostaje na nivou desetina milisekundi i raste blago sa veličinom skupa (od 0.0146 s do 0.0512 s), dok brute-force vrijeme raste višestruko (od 2.25 s do 57.09 s). Dobitak proizlazi iz smanjenja broja poređenja: prosječan broj kandidata (`avg_candidates`) je višestruko manji od N_{docs} (npr. $\approx$ 30 kandidata za 20 000 dokumenata). Metrika `hit_at_topk` dodatno pokazuje da u većim podskupovima LSH češće pronalazi relevantan pogodak među TOPK (0.204 za 20 000 dokumenata), jer raste vjerovatnoća da u kolekciji postoji zaista sličan dokument.

Table 3. Poređenje vremena izvršavanja: LSH naspram brute-force pretrage (TOPK=10, $N_QUERIES = 50$).

N_{docs}	time_lsh_sec	time_bruteforce_sec	speedup_x	avg_candidates	hit_at_topk
1000	0.014643	2.247047	153.451	0.96	0.032
4000	0.020727	10.999752	530.687	3.04	0.092
10000	0.028740	34.231147	1191.046	4.86	0.058
15000	0.037527	45.701160	1217.818	14.88	0.068
20000	0.051169	57.091223	1115.745	29.68	0.204

Ubrzanje LSH pretrage u odnosu na brute-force. Empirijski odnos vremena iz Tabele 3 potvrđuje očekivani trend: LSH postiže ubrzanje od približno 150× do oko 1200× na testiranim podskupovima. Slika 3 vizuelno prikazuje kako ubrzanje raste sa veličinom skupa (brute-force raste približno linearno sa N, dok LSH zadržava nizak trošak zahvaljujući filtriranju kandidata), uz blagi pad na najvećem podskupu zbog porasta prosječnog broja kandidata i varijacija raspodjele po bucketima.

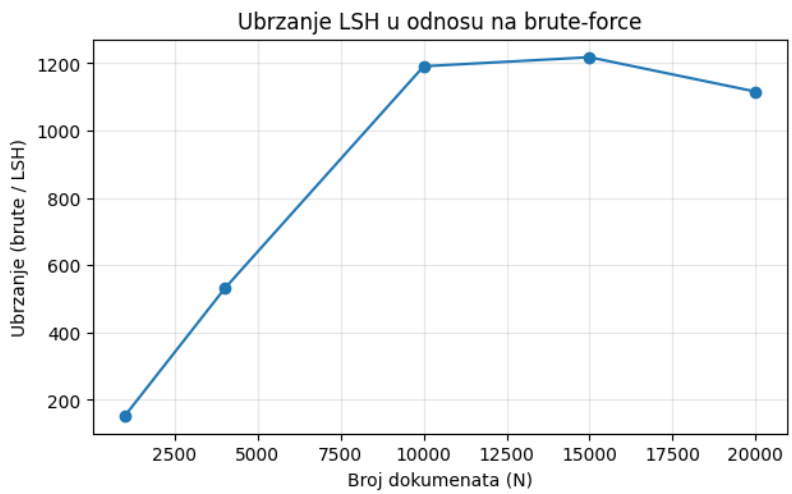

Fig. 3. Ubrzanje LSH pretrage u odnosu na brute-force pri rastu broja dokumenata.

5 Zaključak

U ovom radu implementiran je sistem za pronalaženje sličnih dokumenata zasnovan na MinHash potpisima i Locality-Sensitive Hashing (LSH) indeksiranju, uz predstavljanje dokumenata kao skupova shinglova i mjerenje sličnosti Jaccardovim koeficijentom. Odabrana finalna konfiguracija pokazala se kao stabilan kompromis između broja kandidata i kvaliteta vraćenih rezultata na testiranim podskupovima do 20 000 dokumenata.

Eksperimentalni rezultati potvrđuju da LSH značajno smanjuje broj poređenja u odnosu na brute-force pretragu: vrijeme pretrage ostaje na nivou desetina milisekundi i raste blago s veličinom skupa, dok brute-force raste višestruko (sekunde do desetine sekundi). Empirijski mjerena ubrzanja su reda 10^2–10^3, pri čemu je prosječan broj kandidata višestruko manji od ukupnog broja dokumenata. Istovremeno, metrika *hit_at_topK* pokazuje da se s porastom kolekcije povećava vjerovatnoća da se relevantan pogodak pojavi među prvih K rezultata, što ukazuje da sistem zadržava praktično koristan kvalitet uz vrlo niske troškove pretrage. Kao ograničenje, uočeno je da prosječne stvarne

Jaccard sličnosti mogu biti niske za opšte tekstove, što je očekivano kod velikih i raznovrsnih kolekcija.

5.1 Budući rad i realne primjene

Dodatno unapređenje sistema prvenstveno zahtijeva evaluaciju na znatno većim kolekcijama (npr. stotine hiljada ili milioni dokumenata), gdje bi se jasnije ispitala skalabilnost indeksa i stabilnost metrika. Takvi eksperimenti podrazumijevaju veće računske i memorijske resurse, kao i pažljivije upravljanje pohranom potpisa i bucket struktura. U tom smjeru, korisno je razmotriti distribuirano indeksiranje, inkrementalno ažuriranje (dodavanje novih dokumenata bez ponovne izgradnje cijelog indeksa) te optimizacije pretprocesiranja (npr. filtriranje vrlo čestih shinglova) kako bi se smanjio broj lažnih kandidata.

Pristup zasnovan na MinHash+LSH posebno je primjenjiv u scenarijima gdje je potrebno brzo pronalaziti near-duplicate sadržaj u velikim tokovima podataka. Realne primjene uključuju: (i) deduplikaciju i grupisanje vijesti i web članaka radi detekcije istih/vrlo sličnih objava iz različitih izvora, (ii) detekciju plagijata ili prepisivanja u tekstualnim dokumentima i studentskim radovima kao brzu fazu pretrage kandidata prije detaljnije provjere, (iii) čišćenje korpusa za mašinsko učenje uklanjanjem duplikata i skoro-duplikata, te (iv) identifikaciju ponovljenih opisa proizvoda, oglasa ili korisničkih komentara na platformama sa velikim brojem unosa. U svim navedenim slučajevima, LSH služi kao efikasan filter koji drastično smanjuje prostor pretrage, nakon čega se nad manjim skupom kandidata mogu primijeniti preciznije (ali skuplje) metode poređenja.

References

[1] Andrei Z. Broder. On the resemblance and containment of documents. In *Proceedings of the Compression and Complexity of Sequences*, 1997.

[2] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval.* Cambridge University Press, 2008.

[3] Jiawei Han, Micheline Kamber, and Jian Pei. *Data Mining: Concepts and Techniques.* Morgan Kaufmann, 3rd edition, 2011.

[4] Jure Leskovec, Anand Rajaraman, and Jeffrey D. Ullman. *Mining of Massive Datasets.* Cambridge University Press, 3rd edition, 2020.

[5] Daniel Lemire and Owen Kaser. Strongly universal string hashing is fast. *arXiv preprint arXiv:1202.4961*, 2012.

[6] DTU Compute. Universal hashing. Lecture notes, 2024.